

STA211 – Révision : Arbres de régression/classification et Méta-algorithmes (Bagging & Boosting)

Préparé par Vincent Audigier & Ndèye Niang

Samedi 6 juin 2026

Table des matières

1 Arbres de régression et de classification	2
1.1 Contexte	2
1.2 Généralités	2
1.2.1 Principe	2
1.2.2 Représentations graphiques	2
1.3 Construction d'un arbre	2
1.3.1 Arbres de régression	2
1.3.2 Arbre de classification	7
1.4 Évaluation des performances	9
1.5 Propriétés	9
2 Méta-algorithmes	9
2.1 Introduction	9
2.2 Bagging (Bootstrap Aggregating)	9
2.2.1 Régression	9
2.2.2 Classification	11
2.2.3 Importance des variables	11
2.3 Boosting (AdaBoost)	13
2.4 Comparaison Bagging vs Boosting	16
3 Exemples pratiques	16
3.1 Forêts aléatoires	16
3.2 Extra-Trees	16
3.3 Importance des variables dans les forêts	16
3.4 Parallélisation	16
4 Conclusion	16

1 Arbres de régression et de classification

1.1 Contexte

Les arbres font partie des méthodes supervisées. Leur but est d'expliquer et/ou prédire une variable réponse Y à partir d'un ensemble de variables explicatives $X = (X_1, \dots, X_p)$.

1.2 Généralités

1.2.1 Principe

Les arbres sont des méthodes de **segmentation**. Le principe est de partitionner l'espace des variables explicatives en R régions distinctes $\mathcal{R}_1, \dots, \mathcal{R}_R$. La prédiction pour une observation x est alors :

$$f(x) = \sum_{r=1}^R c_r \mathbb{I}(x \in \mathcal{R}_r)$$

où c_r est une constante (la moyenne des y_i dans la région r pour la régression, ou la classe majoritaire pour la classification).

1.2.2 Représentations graphiques

Un arbre peut être représenté de deux façons :

1. Un arbre de décision avec des noeuds (divisions) et des feuilles (régions terminales)
2. Un partitionnement de l'espace des entrées en régions rectangulaires

1.3 Construction d'un arbre

1.3.1 Arbres de régression

Modèle. Pour une partition donnée, on minimise l'erreur quadratique :

$$\mathcal{C}_{MC} = \sum_{i=1}^n (y_i - f(x_i))^2$$

Recherche d'une partition optimale. On procède par divisions dyadiques récursives. Pour une région, on cherche la variable j et le seuil s qui minimisent :

$$\min_{s,j} \left[\min_{c_1} \sum_{x_i \in \mathcal{R}_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in \mathcal{R}_2(j,s)} (y_i - c_2)^2 \right]$$

où $\mathcal{R}_1(j, s) = \{X \mid X_j \leq s\}$ et $\mathcal{R}_2(j, s) = \{X \mid X_j > s\}$.

Application numérique — Recherche du meilleur seuil

Six clients avec leur âge et leur score de crédit :

Client	Âge	Score
1	23	62
2	35	75
3	45	85
4	28	65
5	50	90
6	32	72

Scission à 30 ans : $\mathcal{R}_1 = \{23, 28\}$, $\hat{c}_1 = 63.5$ $\mathcal{R}_2 = \{35, 45, 50, 32\}$, $\hat{c}_2 = 80.5$

$$MSE = (62 - 63.5)^2 + (65 - 63.5)^2 + (75 - 80.5)^2 + (85 - 80.5)^2 + (90 - 80.5)^2 + (72 - 80.5)^2$$

$$MSE = 4.5 + 213$$

$$MSE = 217.5$$

Scission à 40 ans : $\mathcal{R}_1 = \{23, 35, 28, 32\}$, $\hat{c}_1 = 68.5$ $\mathcal{R}_2 = \{45, 50\}$, $\hat{c}_2 = 87.5$

$$MSE = (62 - 68.5)^2 + (75 - 68.5)^2 + (65 - 68.5)^2 + (72 - 68.5)^2 + (85 - 87.5)^2 + (90 - 87.5)^2$$

$$MSE = 109 + 12.5$$

$$MSE = 121.5$$

Le seuil 40 ans minimise MSE ($121.5 < 217.5$). On retient cette division.

Code Python :

```
1 import numpy as np
2 age = np.array([23, 35, 45, 28, 50, 32])
3 score = np.array([62, 75, 85, 65, 90, 72])
4 for s in [30, 40]:
5     c1, c2 = score[age <= s].mean(), score[age > s].mean()
6     mse = ((score[age <= s] - c1)**2).sum() + ((score[age > s] - c2)**2)
7     .sum()
8     print(f"Seuil {s}: MSE = {mse:.2f}")
```

Code R :

```
1 age <- c(23, 35, 45, 28, 50, 32)
2 score <- c(62, 75, 85, 65, 90, 72)
3 for (s in c(30, 40)) {
4     c1 <- mean(score[age <= s]); c2 <- mean(score[age > s])
5     mse <- sum((score[age <= s] - c1)^2) + sum((score[age > s] - c2)^2)
6     cat(sprintf("Seuil %d: MSE = %.2f\n", s, mse))
7 }
```

Élagage (pruning). Un arbre trop profond souffre de sur-apprentissage. On utilise le **critère coût-complexité** :

$$C_\alpha(A) = \underbrace{\sum_{r=1}^{|A|} \sum_{x_i \in \mathcal{R}_r} (y_i - \hat{c}_r)^2}_{\text{erreur d'ajustement}} + \alpha |A|$$

où $|A|$ est le nombre de feuilles et $\alpha \geq 0$ un paramètre de pénalité.

Application numérique — Élagage

Soit un arbre A avec $|A| = 6$ feuilles et une erreur de 200. Un sous-arbre élagué A' a $|A'| = 3$ feuilles et une erreur de 260.

α	$C_\alpha(A) = 200 + 6\alpha$	$C_\alpha(A') = 260 + 3\alpha$
15	290	305
30	380	350

Pour $\alpha = 15$, A est préféré ($290 < 305$). Pour $\alpha = 30$, A' est préféré ($350 < 380$). Plus α est grand, plus on pénalise la complexité.

Code Python :

```
1 for alpha in [15, 30]:
2     c_a = 200 + alpha * 6
3     c_a2 = 260 + alpha * 3
4     best = "A" if c_a < c_a2 else "A'"
5     print(f"alpha={alpha}: C(A)={c_a}, C(A')={c_a2} -> {best}")
```

Code R :

```
1 for (alpha in c(15, 30)) {
2     c_a <- 200 + alpha * 6
3     c_a2 <- 260 + alpha * 3
4     best <- ifelse(c_a < c_a2, "A", "A'")
5     cat(sprintf("alpha=%d: C(A)=%d, C(A')=%d -> %s\n", alpha, c_a, c_a2,
6         best))
6 }
```

L'algorithme CART construit d'abord l'arbre maximal, puis l'élague en générant une séquence de sous-arbres emboîtés. Le meilleur sous-arbre est choisi par validation croisée ou sur un échantillon de validation.

1.3.2 Arbre de classification

Quand Y est qualitative à M modalités, on utilise des **critères d'impureté** :

— **Erreur de mauvais classement** :

$$\frac{1}{n_r} \sum_{x_i \in \mathcal{R}_r} \mathbb{I}(y_i \neq \hat{y}_i)$$

— **Indice de Gini** :

$$\sum_{m \neq m'} \hat{p}_{rm} \hat{p}_{rm'} = \sum_{m=1}^M \hat{p}_{rm} (1 - \hat{p}_{rm})$$

— **Déviance (entropie)** :

$$- \sum_{m=1}^M \hat{p}_{rm} \log \hat{p}_{rm}$$

où

$$\hat{p}_{rm} = \frac{1}{n_r} \sum_{x_i \in \mathcal{R}_r} \mathbb{I}(y_i = m)$$

Application numérique — Critères d'impureté

Dans une région contenant 4 bons payeurs (Good) et 2 mauvais (Bad) :

$$\hat{p}_G = \frac{4}{6} = 0.6667 \quad \hat{p}_B = \frac{2}{6} = 0.3333$$

Gini :

$$\begin{aligned} G &= \hat{p}_G(1 - \hat{p}_G) + \hat{p}_B(1 - \hat{p}_B) \\ G &= 0.6667 \times 0.3333 + 0.3333 \times 0.6667 \\ G &= 0.4444 \end{aligned}$$

Déviance :

$$\begin{aligned} D &= -(0.6667 \log 0.6667 + 0.3333 \log 0.3333) \\ D &= 0.6365 \end{aligned}$$

Si la région était pure (6 Good, 0 Bad) : $G = 0$, $D = 0$.

Code Python :

```
1 import numpy as np
2 p = np.array([4/6, 2/6])
3 gini = (p * (1 - p)).sum()
4 dev = -(p * np.log(p)).sum()
5 print(f"Gini = {gini:.4f}, Deviance = {dev:.4f}")
```

Code R :

```
1 p <- c(4/6, 2/6)
2 gini <- sum(p * (1 - p))
3 dev <- -sum(p * log(p))
4 cat(sprintf("Gini = %.4f, Deviance = %.4f\n", gini, dev))
```


1.4 Évaluation des performances

On utilise un échantillon test ou la validation croisée pour estimer l'erreur de prédiction (taux d'erreur, AUC, etc.).

1.5 Propriétés

- Avantages : interprétabilité, gestion des variables mixtes, non-parametric
- Inconvénients : variance élevée, instabilité

2 Méta-algorithmes

2.1 Introduction

Les méthodes d'agrégation combinent plusieurs prédictions fournies par différents modèles pour améliorer les performances. Nous présentons deux procédures classiques : le **bagging** et le **boosting**.

2.2 Bagging (Bootstrap Aggregating)

2.2.1 Régression

Le bagging réduit la variance d'un prédicteur instable (comme un arbre). Principe :

1. Constituer B échantillons bootstrap (tirage avec remise)
2. Ajuster un prédicteur sur chaque échantillon
3. Moyenner les B prédictions :

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

La variance des prédictions diminue quand B augmente :

$$\mathbb{V} \left[\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x) \right] = \frac{1}{B} \mathbb{V}[\hat{f}_b(x)] + \frac{B-1}{B} \text{Cov}[\hat{f}_b(x), \hat{f}_{b'}(x)]$$

Application numérique — Échantillons bootstrap

Avec $n = 5$ observations $\{A, B, C, D, E\}$, trois échantillons bootstrap :

Itération	Échantillon bootstrap
1	A, A, B, C, E
2	B, C, D, D, E
3	A, C, E, E, E

Chaque échantillon contient environ $1 - (1 - \frac{1}{n})^n \approx 63\%$ d'observations uniques. Les B arbres sont ajustés sur ces échantillons, puis leurs prédictions sont moyennées.

Code Python :

```
1 from sklearn.utils import resample
2 X = np.array(['A', 'B', 'C', 'D', 'E'])
3 for b in range(3):
4     boot = resample(X)
5     print(f"Bootstrap {b+1}: {boot}")
```

Code R :

```
1 x <- c("A", "B", "C", "D", "E")
2 for (b in 1:3) {
3     boot <- sample(x, replace = TRUE)
4     cat(sprintf("Bootstrap %d: %s\n", b, paste(boot, collapse = " ")))
5 }
```

Erreur OOB (Out-Of-Bag). Les observations non sélectionnées dans chaque bootstrap servent d'échantillon test :

$$\widehat{OOB} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i^{OOB} - y_i)^2$$

2.2.2 Classification

Pour la classification :

- Agrégation des probabilités prédites (préférable)
- Vote majoritaire des classes prédites
- Erreur OOB adaptée au cadre qualitatif

2.2.3 Importance des variables

L'importance d'une variable X^j est mesurée par la **Mean Decrease Accuracy** :

$$VI(X^j) = \frac{1}{B} \sum_{b=1}^B \left(\widetilde{OOB}_b^j - OOB_b \right)$$

où $\widetilde{OOB}_b^j$ est l'erreur OOB après permutation de X^j .

Application numérique — Importance par permutation

Reprenons les 6 clients. On compare les prédictions avant et après permutation de l'âge :

y réel	$\hat{y}$ (original)	$\hat{y}_{\text{perm}}$ (âge permuté)
62	65	68
75	78	78
85	82	82
65	68	65
90	88	88
72	70	72

MSE de base : $\frac{1}{6} [(62 - 65)^2 + (75 - 78)^2 + \dots] = \frac{46}{6} = 7.67$

MSE après permutation : $\frac{1}{6} [(62 - 68)^2 + (75 - 78)^2 + \dots] = \frac{54}{6} = 9.00$

$VI(\text{Âge}) = 9.00 - 7.67 = 1.33$

Code Python :

```
1 from sklearn.metrics import mean_squared_error
2 y = np.array([62, 75, 85, 65, 90, 72])
3 y_pred = np.array([65, 78, 82, 68, 88, 70])
4 y_perm = np.array([68, 78, 82, 65, 88, 72])
5 vi = mean_squared_error(y, y_perm) - mean_squared_error(y, y_pred)
6 print(f"Importance Age: {vi:.2f}")
```

Code R :

```
1 y <- c(62, 75, 85, 65, 90, 72)
2 y_pred <- c(65, 78, 82, 68, 88, 70)
3 y_perm <- c(68, 78, 82, 65, 88, 72)
4 vi <- mean((y - y_perm)^2) - mean((y - y_pred)^2)
5 cat(sprintf("Importance Age: %.2f\n", vi))
```

2.3 Boosting (AdaBoost)

Le boosting construit séquentiellement des prédicteurs adaptatifs.

Algorithme AdaBoost :

1. Initialiser les poids $p_i^{(0)} = 1/n$
2. Pour $b = 1$ à B :
 - Ajuster un classifieur f_b avec les poids $p_i^{(b-1)}$
 - Calculer l'erreur pondérée $e_b = \sum_{i: f_b(x_i) \neq y_i} p_i^{(b-1)}$
 - Calculer $\alpha_b = \log\left(\frac{1-e_b}{e_b}\right)$
 - Mettre à jour les poids : $p_i^{(b)} = \exp(\alpha_b) \cdot p_i^{(b-1)}$ pour les mal classés
 - Normaliser les poids
3. Classifieur final : $\hat{f}(x) = \text{signe}\left(\sum_{b=1}^B \alpha_b \hat{f}_b(x)\right)$

Application numérique — AdaBoost pas à pas

Quatre clients avec une règle simple : score $< 70 \Rightarrow +1$ (Good) :

i	Score	y_i	$f_1(x_i) = \text{signe}(70 - \text{score})$
1	62	+1	+1 ✓
2	85	-1	-1 ✓
3	72	+1	-1 ×
4	65	-1	+1 ×

Itération 1 :

- Erreur pondérée : $e_1 = w_3 + w_4 = 0.25 + 0.25 = 0.50$
- Poids du classifieur : $\alpha_1 = \frac{1}{2} \log \frac{1-0.5}{0.5} = 0$
- Tous les poids restent à 0.25, l'algorithme passe à l'itération suivante.

Itération 2 : Utilisons un stump différent : $f_2(x) = \text{signe}(80 - \text{score})$

- Mal classé : client 2 (score 85, prédit -1) et client 3 (score 72, prédit -1)
- $e_2 = 0.25 + 0.25 = 0.50 \rightarrow$ même situation.

Ajoutons un 5^e client atypique (score 78, $y = -1$, poids initial 0.20) pour que les poids $e \neq 0.5$:

i	Score	y_i	$f_1(x) = \text{signe}(70 - \text{score})$
1	62	+1	+1 ✓
2	85	-1	-1 ✓
3	72	+1	-1 ×
4	65	-1	+1 ×
5	78	-1	-1 ✓

Itération 1 :

- $e_1 = w_3 + w_4 = 0.20 + 0.20 = 0.40$
- $\alpha_1 = \frac{1}{2} \log \frac{1-0.40}{0.40} = \frac{1}{2} \log(1.50) = 0.203$
- Mise à jour : $w_3 = w_4 = 0.20 \times e^{0.203} = 0.245$; $w_1 = w_2 = w_5 = 0.20 \times e^{-0.203} = 0.163$
- Normalisation : diviser par la somme $2 \times 0.245 + 3 \times 0.163 = 0.979$
- $w = [0.167, 0.167, 0.250, 0.250, 0.167]$

Itération 2 : $f_2(x) = \text{signe}(75 - \text{score})$ classe correctement le client 3.

- $e_2 = w_4$ (client 4 toujours mal classé si score < 75) = 0.250
 - $\alpha_2 = \frac{1}{2} \log \frac{1-0.25}{0.25} = 0.549$
- Classifieur final : $\hat{f}(x) = \text{signe}(0.203 f_1(x) + 0.549 f_2(x))$.

Code Python :

```
1 import numpy as np
2 score = np.array([62, 85, 72, 65, 78])
3 y      = np.array([1, -1, 1, -1, -1])
4 n = len(score); w = np.ones(n) / n
5 for b in range(2):
6     pred = np.where(score < 70, 1, -1)
7     if b == 1: pred = np.where(score < 75, 1, -1)
8     e = w[pred != y].sum()
9     alpha = 0.5 * np.log((1 - e) / max(e, 1e-15))
10    w[pred != y] *= np.exp(alpha)
11    w[pred == y] *= np.exp(-alpha)
12    w /= w.sum()
13    print(f"B={b+1}: e={e:.4f}, alpha={alpha:.4f}, w={np.round(w,4)}")
```

Code R :

```
1 score <- c(62, 85, 72, 65, 78)
2 y      <- c(1, -1, 1, -1, -1)
3 n <- length(score); w <- rep(1/n, n)
4 for (b in 1:2) {
5     if (b == 1) pred <- ifelse(score < 70, 1, -1)
6     if (b == 2) pred <- ifelse(score < 75, 1, -1)
7     e <- sum(w[pred != y])
8     alpha <- 0.5 * log((1 - e) / max(e, 1e-15))
9     w[pred != y] <- w[pred != y] * exp(alpha)
10    w[pred == y] <- w[pred == y] * exp(-alpha)
11    w <- w / sum(w)
12    cat(sprintf("B=%d: e=%.4f, alpha=%.4f\n", b, e, alpha))
13    cat("  w:", round(w, 4), "\n")
14 }
```

2.4 Comparaison Bagging vs Boosting

Caractéristique	Bagging	Boosting
Nature	Aléatoire (bootstrap)	Adaptatif, déterministe
Échantillon	Bootstrap (n tirages avec remise)	Complet (poids adaptatifs)
Agrégation	Poids égaux	Poids selon la qualité
Biais/Variance	Réduit la variance	Réduit biais et variance
Parallélisation	Oui	Non (séquentiel)
Sur-apprentissage	Non (robuste au bruit)	Oui (si trop d'itérations)

TABLE 1 – Comparaison du bagging et du boosting

3 Exemples pratiques

3.1 Forêts aléatoires

Les forêts aléatoires ajoutent au bagging un tirage aléatoire des variables candidates à chaque noeud : L variables parmi p (souvent $L \approx \sqrt{p}$ pour la classification, $p/3$ pour la régression).

3.2 Extra-Trees

Les Extra-Trees (Extremely Randomized Trees) vont plus loin en tirant aléatoirement les seuils de coupure. Ils ne font pas d'échantillonnage bootstrap, ce qui augmente le biais mais diminue davantage la variance.

3.3 Importance des variables dans les forêts

Deux mesures :

- **Mean Decrease Accuracy** : par permutation
- **Mean Decrease Impurity** : baisse d'impureté (Gini) moyennée sur tous les arbres

3.4 Parallélisation

Le bagging est parallélisable (les B échantillons sont indépendants). Le boosting est séquentiel.

4 Conclusion

Les arbres sont des méthodes interprétables mais instables. Les méta-algorithmes (bagging, forêts aléatoires, boosting) permettent d'en améliorer les performances prédictives au détriment de l'interprétabilité (partiellement retrouvée via l'importance des variables). Le choix entre bagging et boosting dépend du contexte (bruit dans les données, besoin de parallélisation, compromis biais-variance).